



Curso de Python

Unidad 1. Introducción al lenguaje Python.

ÍNDICE

Tabla de contenido

Presentación y objetivos	4
1. Introducción.....	5
1.1 Definición	5
1.2 Un poco de historia en los lenguajes de programación.....	6
1.3 Clasificación de los lenguajes de programación.....	9
1.3.1 Clasificación Histórica	9
1.3.2 Clasificación por paradigmas	11
1.3.3 Clasificación por propósitos	13
1.4 Historia de Python.....	13
1.4.1 Utilización de Python.....	16
1.4.2 Python en cifras.	18
1.5 Pero ¿por qué programar?	18
1.6 Filosofía de Python.....	20
2 Instalación de herramientas de desarrollo.....	21
2.1 Versión de Python e instalación.	21
2.2 Instalación de un editor	23
2.3.1 Instalación de Thonny	23
2.3 Nuestro primer programa	25
2.4 Hola mundo!.....	28
2.5 Recibiendo datos.....	30
2.6 Probando todo en replit.com	34

2.7 Python y más allá	38
3 Resumen.....	40

Unidad 1 – Introducción

Presentación y objetivos

Como en todo curso, en esta unidad vamos a realizar una introducción sobre qué es Python, su clasificación dentro de los lenguajes de programación, su historia o qué empresas lo usan, entre otros.

También realizaremos nuestro primer programa (quizá te suene 'Hola Mundo') y dónde podremos realizar nuestros programas. Una cosa interesante es que hay multitud de herramientas gratuitas para poder programar nuestras aplicaciones.

Además, si tienes conocimientos de inglés, a lo largo del curso te pondremos enlaces a distintos libros (libres y gratis) para aumentar y afianzar tus conocimientos.



Objetivos

En esta unidad aprenderás a:

- Qué es Python.
- Primeras herramientas que utilizaremos.
- Programar nuestra primera aplicación en Python.

1. Introducción

En estos momentos te encuentras comenzando un curso sobre un lenguaje de programación llamado Python. Pero antes de todo empezaremos conociendo qué es un lenguaje de programación, qué tipos hay, distintas clasificaciones, etc.

1.1 Definición

Un lenguaje de programación es un lenguaje formal (tiene una serie de reglas) que nos proporciona una serie de instrucciones o secuencia de órdenes, denominados algoritmos, que nos permite controlar el comportamiento físico o lógico de un ordenador. Así podremos ejecutar diferentes tareas u obtener diversas clases de datos.

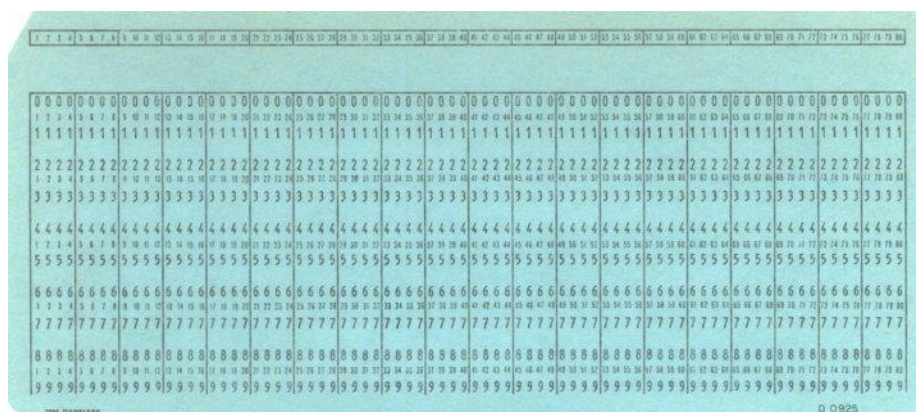


Los comienzos de la programación no fueron nada fáciles Fuente: Wikimedia.org

En la imagen anterior puedes ver cómo se empezó a programar. No todo era tan sencillo como ahora, con teclado, pantalla, etc. La programación consistía en escribir los programas en unas llamadas tarjetas perforadas, donde o tenías el manual delante o no era nada intuitivo saber lo que hacía el programa.

La tarjeta se pasaba “al ordenador”, la ejecutaba y se obtenía el resultado. Pero no en pantalla, ya que no existía.

Además, el uso de estos primeros ordenadores estaba muy limitado, por lo que había que pedir turno de días para poder ejecutar nuestros programas. El problema venía que si nuestro programa tenía un error, debíamos esperar al siguiente turno para poder corregirlo.



Fuente: Wikimedia.org

En estas tarjetas perforadas se escribían los programas. Como puedes apreciar, la programación de entonces era poco intuitiva.

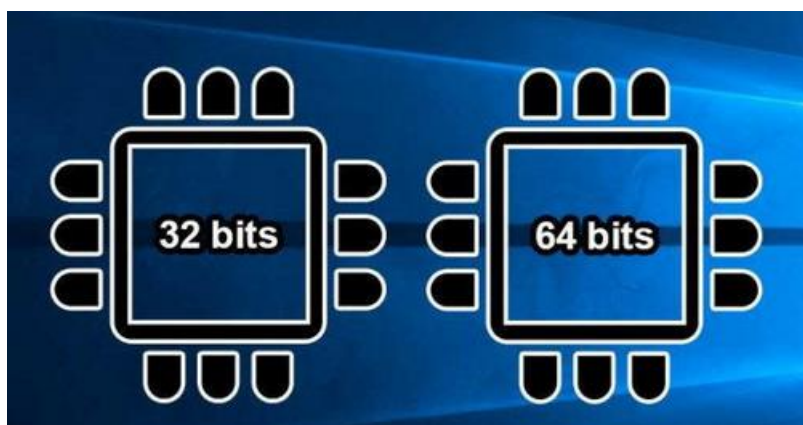
1.2 Un poco de historia en los lenguajes de programación

El código que entiende una computadora se denomina **código máquina**. Es decir, que los programas que escribimos no los ejecuta directamente el ordenador. Sufre un proceso previo a la ejecución.

Porque el código máquina son únicamente 0 y 1. Aunque esto está empezando a cambiar con los ordenadores cuánticos. Es otra revolución en la informática.

Enlace: [Línea del tiempo de los lenguajes de programación](#)

Al principio cada máquina incluía especificidades que era preciso tener en cuenta cuando programábamos (como la arquitectura del ordenador o capacidad), pero más tarde esto se ha dejado de tener en cuenta y otras han ido evolucionando.



Fuente: xataka

Una de las elecciones era si debíamos programar para 32 o 64 bits. Dependiendo de la arquitectura interna del ordenador, es decir, cómo estaba construida la CPU. Los bits hacen referencia a la longitud de la instrucción que podían ejecutar. 64 bits tenían más potencia que 32. Aunque hoy en día ese dilema está superado ya que prácticamente no existen ordenadores de 32 bits (lo mismo pasa con los móviles).

La evolución del código máquina fue **el lenguaje ensamblador**. En este caso ya no estaba escrito en 0 o 1. Se utilizaban algunas palabras para describir las operaciones. Veamos un ejemplo:

En código máquina, para sumar dos números usábamos la instrucción 10101100. Pero en ensamblador se utilizaba la letra A (en inglés sumar se escribe add). Lo que hacía los lenguajes algo más entendibles.

Aquí tienes un lenguaje hecho en ensamblador, algo más sencillo que en código máquina, pero aún así difícil de entender:

```
ORG 100h
mov ax, 10 ;AX=10
mov bx, 00F9h ;BX=0xF9
inc bx ;BX++
add ax, 4 ;AX=AX+4
mov cx, 45 ;CX=45
sub cx, cx ;CX=CX-CX
ret
```

En este lenguaje se expresaba al ordenador al 100% y había que tener en cuenta sus características.

Los siguientes lenguajes ya son algo más entendibles y dedicados a fines concretos (hablamos de finales de los años 50):

Fortran: Son las siglas de Formula Translating. Es de propósito general, aunque orientado al cálculo numérico y la computación científica.

COBOL (*Lenguaje Común Orientado a Negocios, en castellano*): Como dice su acrónimo, está orientado a los negocios y a las finanzas. Como curiosidad, aunque el lenguaje tiene más de 70 años, todavía se utilizan programas realizados en COBOL. Debido también a la rapidez de uso de dichos programas, ya que no se utiliza el ratón y hay usuarios muy rápidos en su uso.

Los siguientes lenguajes se denominan de alto nivel, ya más cercanos al lenguaje humano. Como ejemplo de ello tenemos **BASIC** (*Código simbólico de instrucciones de propósito general para principiantes, en castellano*). Fue creado en el Dartmouth College en New Hampshire, Estados Unidos. Ha inspirado a numerosos lenguajes muy utilizados hoy en día, como Microsoft Visual Basic de Microsoft o Gambas en GNU/Linux.

Hoy en día existen decenas de lenguajes de programación. Muchos de ellos orientados a ciertos aspectos de la informática, ya sean páginas web, servidores, multimedia o cualquier cosa que se nos ocurra.

Una vez vista una ligera historia de los lenguajes de programación (no es el propósito principal de este curso), haremos una clasificación de los lenguajes dependiendo del criterio con que se haga.

1.3 Clasificación de los lenguajes de programación

Como se ha dicho en el punto anterior, haremos varias clasificaciones de los lenguajes. Empezamos por una clasificación dependiendo de cuándo se crearon.

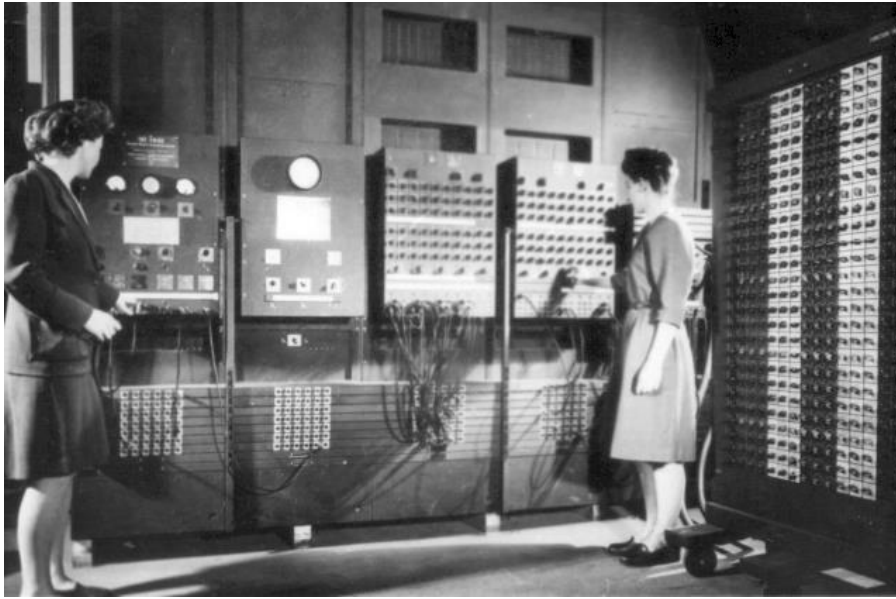
Al final de las diferentes clasificaciones, indicaremos dónde se ubica Python en cada una de ellas

1.3.1 Clasificación Histórica

En esta primera, se hace según la evolución de los ordenadores, porque no han sido tal y como los conocemos actualmente. Tenemos 5 generaciones en total.

Primera Generación

Programados directamente en código máquina (acuérdate de los 0 y 1). Se denominan de bajo nivel, porque sus instrucciones actúan directamente sobre el hardware de la máquina y están condicionadas por la estructura física. Ya hemos visto que es difícil programar con este tipo de lenguajes. No quiere decir que sean poco potentes, sino que son difíciles de programar.



Fuente: Wikimedia.org

ENIAC, uno de los primeros ordenadores, que era programado con lenguajes de primera generación.

Segunda generación

Los lenguajes se hacen más legibles, como el ensamblador. Se pueden emplear macros (conjunto de instrucciones que se utilizan como un todo) y estructuras de datos más complejas.

Tercera generación

Ya son de alto nivel, más cercano al lenguaje humano e independientes de la máquina.

Son de ámbito de aplicación más general, ya no con un único propósito. Además, comienzan a utilizarse las bibliotecas, para poder reutilizar el código.

Ejemplos de estos, que hoy en día son muy utilizados: C, Fortran, Smalltalk, Ada, C++, C#, Cobol, Delphi, Java y PHP.

Cuarta generación

Se consideran a ciertas herramientas en las que se combinan piezas preprogramadas. Algunos autores piensan que estos no son lenguajes en sí. Otros

consideran que estos lenguajes son los llamados de Programación Orientada a Objetos.

Además, acceden a bases de datos, se aumenta la productividad y hay un menor cansancio del programador, ya que muchas cosas que quieren hacer ya están hechas.

Ejemplos de esta generación: Visual Basic o SQL.

Quinta generación

Son muy actuales y se refiere a un conjunto de lenguajes que resuelven problemas utilizando la Inteligencia Artificial. Se utilizan para resolver problemas complejos.

1.3.2 Clasificación por paradigmas

Un paradigma consiste en un método para organizar las tareas que debe realizar un programa.

El principal paradigma utilizado hoy en día, ya se ha comentado en la clasificación anterior, es el de Orientación a objetos. Este punto se explicará en el punto 3 del curso.

Si clasificamos un lenguaje por paradigmas nos resultará una clasificación demasiado extensa por lo que la resumiremos.

Programación imperativa

Le damos al ordenador instrucciones para indicarle lo que debemos realizar, basándose normalmente en algoritmos.

Ejemplos: C, BASIC y Pascal.

Programación Orientada a Objetos

Basada en la imperativa, pero encapsula (agrupa) en objetos tanto variables como funciones.



Fuente: edd

Programación dirigida por eventos

Tanto la estructura como la ejecución de los programas van determinados por los sucesos que ocurran en el sistema, definidos por el usuario.

Por ejemplo, indicamos qué ocurre cuando hacemos clic en cierta parte del programa o qué responder cuando ocurre un error.

Programación declarativa

Se basa en describir el problema declarando propiedades y reglas que deben cumplirse. Suelen ser utilizados para razonar matemáticamente.

Programación funcional

Basada en la definición los predicados y es de corte más matemático. Python podríamos encuadrarlo en este tipo de programación.

Programación multiparadigma

Lo más lógico es que un lenguaje no se pueda encuadrar en algún tipo de paradigma. Es decir, puede ser Orientado a objetos e imperativo.

Aunque algunos autores ubican a Python en este paradigma.

1.3.3 Clasificación por propósitos

Propósito general

Con el lenguaje no se pretende resolver un tipo de problema específico sino que intenta abarcar el mayor número de ellos. La mayoría de lenguajes son un buen ejemplo

Propósito específico

Con este tipo de lenguajes se tratan un tipo de problemas de la forma más eficiente posible.

Como ejemplo tenemos LISP, orientado a matemática y al trabajo con listas.

En nuestro caso, Python lo podemos encasillar en:

- De cuarta generación
- Lenguaje de alto nivel
- Imperativo y Orientado a objetos
- De propósito general.

1.4 Historia de Python

No sé si tendrás experiencia en programación, pero cuando todo el mundo estudia un lenguaje nuevo de programación, siente una especie de temor por si el lenguaje será muy complicado de aprender. En nuestro caso, Python tiene una curva de aprendizaje amigable. Así que, fuera miedos y adelante.

Bueno, comencemos por el principio. Si buscas información sobre Python te dirán que es un lenguaje de alto nivel. ¿Qué quiere decir esto? Quiere decir que cuando

escribamos los algoritmos (conoceremos más adelante lo que son) , el lenguaje que utilizaremos está muy próximo a nuestra capacidad cognitiva. Por ejemplo, si queremos mostrar algo por pantalla usaremos la **orden print**, que nos resultará bastante intuitivo. También es imperativo (como hemos visto anteriormente) , orientado a objetos y basado en fuentes abiertas.

Siguiendo con un poco de historia, el lenguaje Python fue desarrollado por Guido van Rossum, a finales de los 80. Como se aburría, unas Navidades empezó a darle continuidad al lenguaje ABC, del cual había sido parte del conjunto de desarrolladores. Para ello cogió lo mejor de varios lenguajes y los fusionó. Ya que el lenguaje ABC era fácil de utilizar, pero el hardware de la época no facilitaba su uso, de ahí que el lenguaje ABC no triunfó como se esperaba.

La idea de Guido era crear un lenguaje fácilmente entendible sin necesidad de conocer órdenes extrañas, caracteres hexadecimales o ceros y unos. Y creo que lo consiguió.

Como curiosidad, decir que Python fue originalmente diseñado para el sistema operativo Amoeba (nada de Windows ni Linux)



Logo actual de Python Fuente: Python.org

Entonces, cronológicamente podemos decir que la implementación de Python empezó en Diciembre de 1989, la versión 1.0 llegó en enero de 1994 y actualmente se encuentra en la versión 3.11.4



Fuente: somoslibres.org

Este es Guido en la actualidad. Pero conozcamos un poco más de la vida de Guido.

Nació el 31 de enero de 1956 en los Países Bajos y es conocido por ser el autor del lenguaje de programación Python, como ya hemos comentado.

Es la primera persona en obtener el título BDFL (*Benevolent Dictator for Life*), cuyo título es otorgado a personajes importantes en el mundo del código abierto. Haz clic [aquí](#) si quieres conocer más sobre el mundo del código abierto, muy relacionado con Python.

Guido es el que se encarga de tomar las decisiones finales en las nuevas versiones de Python. Aunque algunos le acusan de ser demasiado conservador, ya que siempre quiere que las nuevas versiones sean compatibles con las actuales. Aunque en el año 2018 se retiró de los procesos de la toma de decisiones en las nuevas versiones.

Como curiosidad, decir que es Licenciado en Matemáticas y Computación por la Universidad de Ámsterdam, en el año 1982.



Guido dando una charla Fuente: Getty Images.

Aparte de trabajar en la Universidad de Nueva York, también trabajó en Google y en Dropbox.

Cuando le llegó la hora de la jubilación, decidió entrar a trabajar en Microsoft. Una de las cosas que ha logrado allí es aumentar la velocidad de algunas partes de Python entre un 10 y un 60%, corrigiendo ciertas partes del diseño.

Si tienes curiosidad por el nombre, Guido se lo puso en homenajes a los Monthly Python, después de leer algunos diálogos suyos. Aunque muchas veces se representa como una serpiente pitón.

1.4.1 Utilización de Python

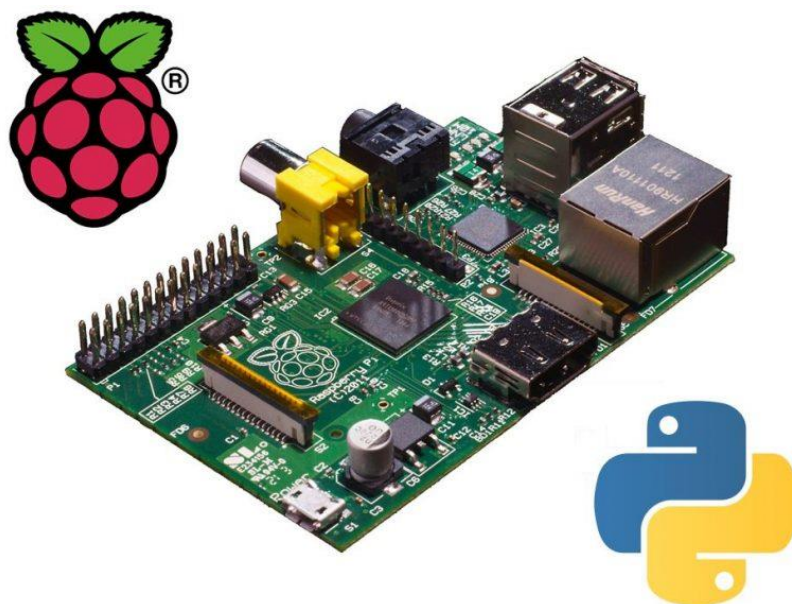
A estas alturas del curso te preguntarás, pero ¿quién usa Python hoy en día? Quizá si te digo las palabras Alexa o Siri te suenen de algo. En el caso de estas empresas, utilizan Python por su tecnología para el reconocimiento de la voz. Que es uno de los puntos donde destaca Python.

Por si esto fuera poco, multitud de empresas conocidas también utilizan Python.

- Netflix: aprendizaje automático, automatización o seguridad.
- Instagram: autenticación o mejora de las fotos en los selfies.
- Spotify: análisis masivo de datos o servicios de backed (procesos internos de Spotify)
- Disney: renderizado de sus películas.
- Facebook: lo utilizan para filtrar los comentarios, usando machine learning.

Hoy en día Python destaca en la ciencia de datos (Amazon sabe mucho de eso), blockchain, análisis de datos o aprendizaje de las máquinas (machine learning). También se utiliza mucho en análisis financieros. Para realizar muchas tareas de forma automática. Existen librerías (programas ya hechos) que nos pueden ayudar en la extracción y visualización de datos financieros. [Haz clic aquí para ver un breve resumen](#)

En cuanto a su aprendizaje, en Inglaterra, desde la edad escolar, los alumnos aprenden a usar Python. Para ello utilizan la raspberry pi (un ordenador de bajo coste). Es decir, con un ordenador de unos 50€, un teclado, un ratón y nuestra televisión, podemos aprender y utilizar Python.



Fuente: prometec.net

Raspberry y Python, una buena combinación

¿Todo esto no te parece una buena razón para aprender Python?

1.4.2 Python en cifras.

Según multitud de webs, Python se encuentra entre los 3 lenguajes más utilizados a nivel mundial. Y en la mayoría de ellas ocupa el primer lugar. En los últimos 5 años, en 3 de ellos ha ocupado el primer lugar de utilización. Alternando el primer puesto con Java. Por lo que si lo aprendes puede ser que no te falte trabajo (e incluso podrías teletrabajar).

(Fuente: <https://www.tiobe.com/tiobe-index/>)

Entre los programadores, es utilizado por el 12.5% de ellos, aumentando su uso este año 2023. Por lo que no serás un bicho raro si lo utilizas para programar.

Jun 2023	Jun 2022	Change	Programming Language	Ratings	Change
1	1		 Python	12.46%	+0.26%
2	2		 C	12.37%	+0.46%
3	4	▲	 C++	11.36%	+1.73%
4	3	▼	 Java	11.28%	+0.81%
5	5		 C#	6.71%	+0.59%

Los 5 primeros puestos Fuente: www.tiobe.com

Debido a todos estas cifras, lógicamente Python se encuentra entre los lenguajes más demandados por parte de las empresas. Y si algo es altamente demandado quiere decir que tendrá su recompensa quien entre a trabajar como programador de Python.

1.5 Pero ¿por qué programar?

Una vez elegido el lenguaje que aprenderemos (por eso estás en este curso), te daré unas razones para que conozcas lo importante que es saber programar.

1.- Potencias la creatividad, es decir, fomentas tu lado más creativo, una habilidad que vas a necesitar en todas las facetas de tu vida.

2.- No solo estimulas tu pensamiento lógico y estructurado, si no que entiendes la lengua que habla la tecnología. En muchos países ya se imparte en la Educación Primaria y Secundaria asignaturas donde se incluye la programación (quizá te suene [Scratch](#) o [Appinventor](#))

3.- Ganas autonomía en el terreno profesional, ya que puedes solucionar problemas con la tecnología que aparecen en tu día a día.

4.- Puedes programar desde casa, por lo que se fomenta el teletrabajo.

5.- Para muchas personas puede llegar a ser divertido, como un puzle en el que debes encajar ciertas piezas de una forma concreta para conseguir el resultado deseado (que el programa resuelva un problema concreto).

En muchos libros leerás que programar es como escribir una receta. Consiste en una serie de pasos que hay que seguir en un orden en concreto:

Algoritmo de un pastel:

Tener un bol

Harina = 100

Mantequilla = 50

Leche = 100

Añadir al bol (harina,
mantequilla, leche).

Pastel.cocinar (bol)



Fuente: wikipedia.org

A la izquierda tenemos los pasos a seguir y a la derecha el resultado

1.6 Filosofía de Python

A continuación, tienes lo que se denomina la filosofía de Python (que es bastante similar a la filosofía de Unix). El código que siga estos principios se denomina como pythónico. Fue escrito por Tim Peters (un experimentado programador de este lenguaje) y es aplicable a cualquier proyecto de programación. Éste es el zen de Python:

- Bello es mejor que feo.
- Explícito es mejor que implícito.
- Simple es mejor que complejo.
- Complejo es mejor que complicado.
- Plano es mejor que anidado.
- Disperso es mejor que denso.
- La legibilidad cuenta.
- Los casos especiales no son tan especiales como para quebrantar las reglas.
- Lo práctico gana a lo puro.
- Los errores nunca deberían dejarse pasar silenciosamente.
- A menos que hayan sido silenciados explícitamente.
- Frente a la ambigüedad, rechaza la tentación de adivinar.
- Debería haber una —y preferiblemente solo una— manera obvia de hacerlo.
- Aunque esa manera puede no ser obvia al principio a menos que usted sea holandés.
- Ahora es mejor que nunca.
- Aunque *nunca* es a menudo mejor que *ya mismo*.
- Si la implementación es difícil de explicar, es una mala idea.
- Si la implementación es fácil de explicar, puede que sea una buena idea.
- Los espacios de nombres (*namespaces*) son una gran idea. ¡Hagamos más de esas cosas!

No vamos a explicar cada apartado anterior porque sería muy extenso. Estos principios son tan prácticos que hacen que incluso existan libros de Python para niños y, como hemos dicho anteriormente, se enseñe incluso en los colegios. Resumiendo dice que no hay que complicarse la vida programando, hay que hacerlo todo sencillo, comprobar todo lo que se hace, etc

Si quieres una explicación más exhaustiva visita esta página: [Explicación](#)

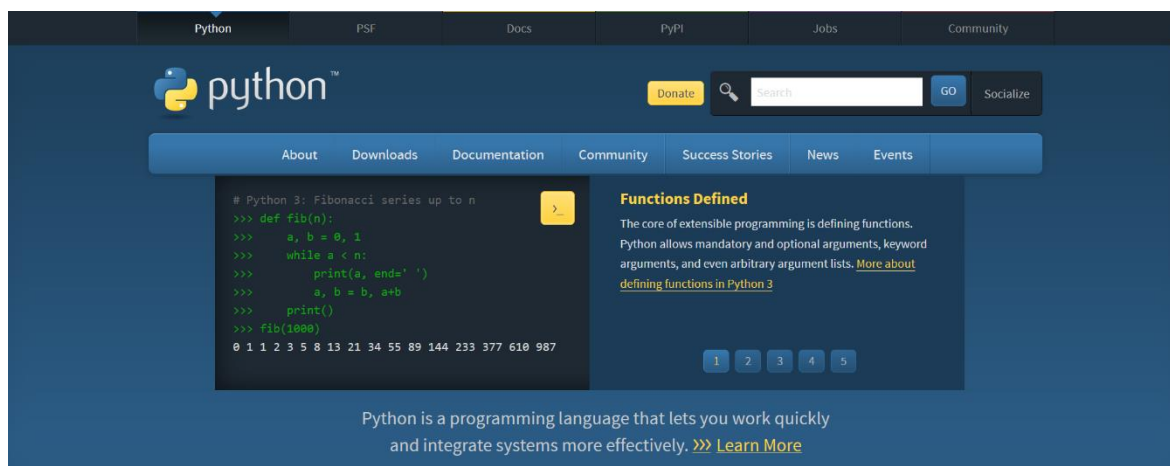
2 Instalación de herramientas de desarrollo.

Ahora que ya conocemos lo que es un lenguaje de programación (y más en concreto Python) es hora de ponernos manos a la obra y dejar los conocimientos teóricos.

¿Qué debo tener para poder aprender Python? En este punto aprenderemos las herramientas (programas) que debemos instalar en nuestro ordenador para poder empezar a aprender a programar en Python. Aunque no es necesario instalar ninguna herramienta de manera obligatoria. Existen páginas web donde podemos realizar nuestras aplicaciones en Python y probarlas. Pero eso lo veremos al final de esta unidad.

2.1 Versión de Python e instalación.

A la hora de redactar este curso, la última versión de Python estable (quiere decir que ha sido probada y no tiene fallos) es la 3.11.4. A la hora de ponernos a programas, tenemos versiones para Windows, Linux, macOS y otros, aunque para poder seguir el curso utilizaremos la versión de Windows.



Página inicial de la web de Python.

El primer paso que debemos realizar es descargar la última versión:

<https://www.python.org/ftp/python/3.11.4/python-3.11.4-amd64.exe>

La descarga del archivo ocupa, aproximadamente, unos 24 megabytes.



Una vez descargado, hacemos doble clic sobre el archivo y comenzamos la instalación:

Hacemos clic en Install Now y en la pantalla que aparece a continuación, le damos a Permitir los cambios.

Y si todo va bien, después de unos segundos, deberíamos tener nuestra instalación de Python terminada.

2.2 Instalación de un editor

Una vez que tenemos instalado el intérprete del lenguaje Python, solo nos queda instalar un editor para poder escribir nuestros programas. El propio bloc de notas podría servirnos, pero iremos un poco más allá, ya que es demasiado simple.

Aunque el propio Python contiene un editor para poder escribir nuestros programas.

Existen versiones sencillas de editores, como Notepad++ o Thonny (que será el que utilizemos en estas primeras unidades).

Hay otras más completas, como Eclipse, PyCharm, Visual Studio Code o Spyder. E incluso algunas son de pago, como Sublime Text. En este caso, estas versiones suelen ser utilizadas por programadores más experimentados.

2.3.1 Instalación de Thonny

Como bien dice en el título de la página web de Thonny, es el IDE de Python para principiantes. Por si no lo sabías, IDE significa Integrated Development Environment (Entorno de Desarrollo Integrado). Conjunto de aplicaciones que contiene todo para poder programar. Normalmente existe un IDE para cualquier lenguaje. Aunque los IDEs suelen ser multilenguaje, es decir, con el mismo IDE podemos programar en varios lenguajes.

Ahora vamos a descargarnos el programa y a instalarlo.

Existen dos versiones, la instalable y la portable (no hace falta instalarla). En este curso utilizaremos la instalable y la versión de Windows 64 bits, aunque también existen versiones para Mac y Linux. Dejamos la libertad al seguidor del curso para utilizar la que prefiera.

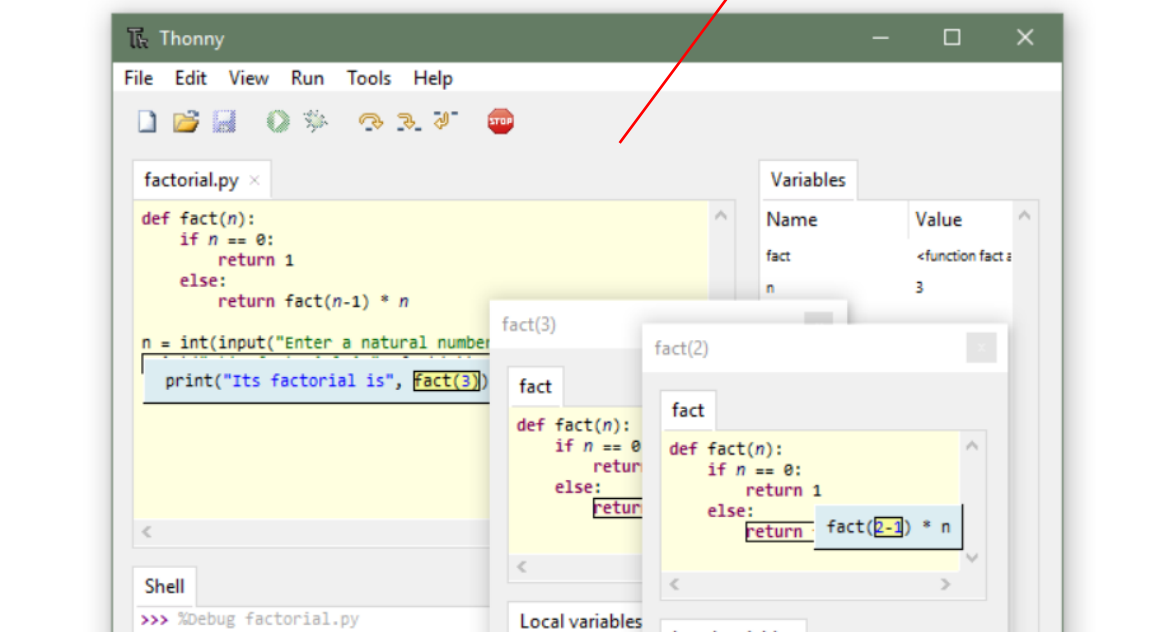
Thonny 4 is dedicated to Ukraine fighting the Russian invasion.
🇺🇦 Please [support Ukraine!](#) 🇺🇦

Thonny

Python IDE for beginners



Download version [4.1.1](#) for
[Windows](#) • [Mac](#) • [Linux](#)



Ahí debemos hacer clic para descargarnos la última versión, la 4.1.1

Thonny

Python IDE for beginners

Puedes hacer clic en el logo para
descargar el programa en la versión de
Windows 64 bits

Ya solo nos queda hacer doble clic y lo típico en la instalación de un programa en Windows, Siguiente, Install, etcétera. Después de unos segundos, clicamos en Finish y ya lo tenemos instalado.



Presta atención

Si el estudiante del curso no quiere instalar nada, existen webs como replit.com , donde podremos escribir nuestro código y probarlo.

Si quieres, investiga y prueba tu primer programa en replit.com después de haberlo hecho con Thonny

2.3 Nuestro primer programa

Una vez instalado nuestro editor de Python, ya estamos en disposición de hacer nuestro primer programa. Es el típico que se realiza cuando se empieza a aprender un lenguaje nuevo de programación. Y se llama Hola Mundo. Incluso tiene una entrada en la Wikipedia: [Hola Mundo](https://es.wikipedia.org/wiki/Hola_Mundo)



Fuente: freeCodeCamp

Antes de nuestro Hola Mundo, dejo un par de frases para la reflexión, para cuando nos lleguen momentos de desesperación en la creación de nuestros programas:

Lo MEJOR de la programación es que hace lo que dices.

Lo PEOR de la programación es que hace lo que dices.

¿Por qué lo mejor? Porque un programa no se vuelve loco y hacen lo que les viene en gana, sino que siguen las instrucciones que nosotros les hemos dado. Lo cual es algo lógico.

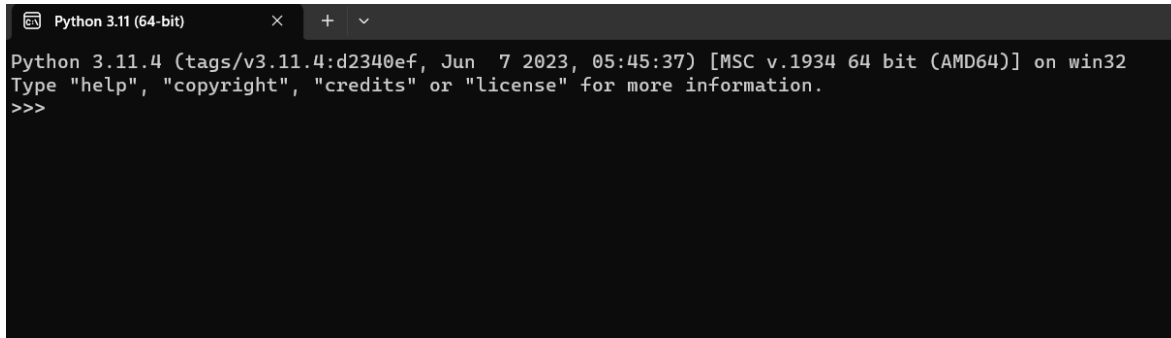
¿Por qué lo peor? Porque nuestro editor que utilicemos no nos ayuda en adivinar qué es lo que queremos hacer con nuestro programa, hay que indicárselo instrucción a instrucción. Aunque esto último está cambiando con la Inteligencia Artificial y con webs que se dedican a hacer de forma automática partes de nuestra aplicación.

Porque quizá nosotros buscamos la solución del problema de una determinada forma y le estamos dando las órdenes incorrectas a nuestro intérprete para la resolución. Y puede ser que a veces lleguemos a un callejón sin salida y tengamos que buscar otros métodos.

Pero vayamos manos a la obra.

Aunque si queremos, no es necesario utilizar Thonny al principio, ya que Python incorpora un intérprete donde podemos escribir nuestras instrucciones.

Para entrar a este intérprete, hacemos clic en Inicio en Windows y escribimos Python. Nos debe aparecer una ventana tal que ésta:

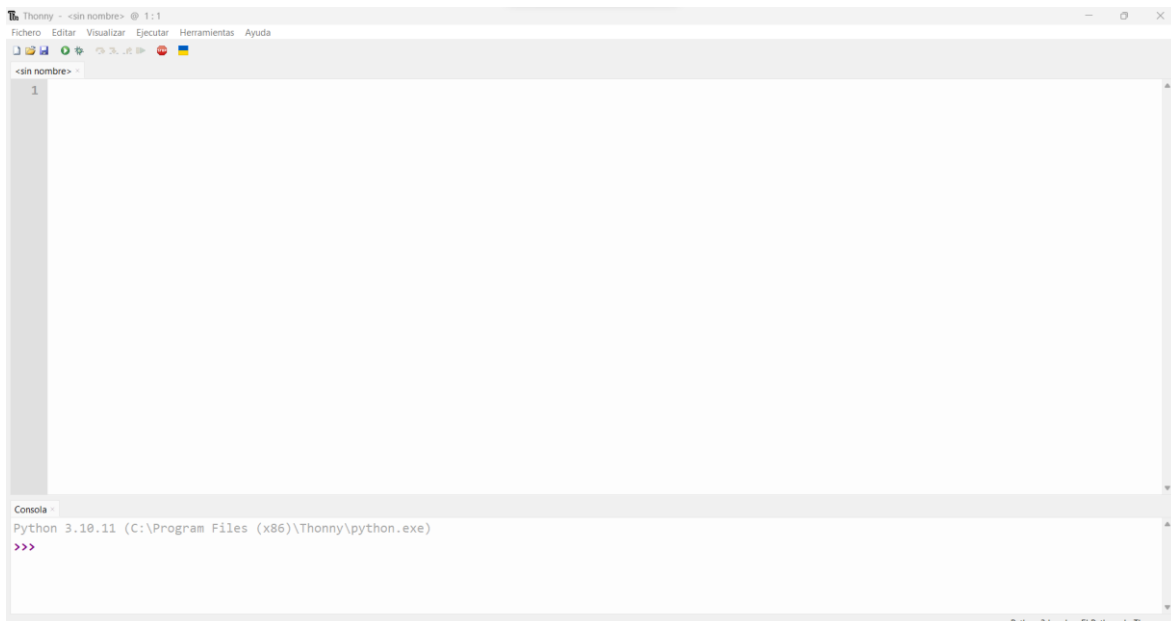


Fuente propia

Pero nuestro primer ejemplo lo haremos con Thonny, aunque también puedes probarlo ahí. Para cerrarlo, escribimos `exit()` o pulsamos `Ctrl + Z` e `Intro`

Pero bueno, sigamos con nuestro ejemplo `Hola Mundo`.

Ejecutamos nuestro editor Thonny y nos aparecerá una ventana como ésta:



Éste será el IDE que utilizaremos en las primeras unidades del curso.

En la parte central escribiremos nuestro código y en la parte inferior (donde pone consola), veremos el resultado de nuestro programa (si todo ha ido bien). Aunque este modo se puede modificar mediante el menú Visualizar.

Porque de momento, nuestra aplicación que programemos no utilizará ventanas, botones, etc. El resultado lo veremos en lo que se denomina Modo Consola, que es como se programaba hace años.

Pero pongámonos manos a la obra.

2.4 Hola mundo!

Escribimos en Thonny lo siguiente:

```
print hola mundo
```

print es la orden que se utiliza para mostrar información por pantalla (porque como dijimos en puntos anteriores Python es un lenguaje de alto nivel y se aproxima mucho al lenguaje humano). Se escribe print ya que la mayoría de lenguajes fueron creados para la utilización del máximo posible de usuarios y es algo que se entiende.

Una vez escrito nuestra primera orden (esto ya se considera un programa en sí) pulsamos F5 o el botón , que hace lo mismo.

Si pensabas que esto de programar es coser y cantar, veremos que tiene sus inconvenientes. Ya tenemos nuestro primer mensaje de error.

En cualquier idioma o lenguaje tienes:

- Lo que quieres decir
- La forma de decirlo

... y puede haber errores en ambas cosas.

Analicemos qué ha pasado.

Al igual que cuando hablamos, debemos cumplir unas reglas. Por ejemplo: Si quiero que me pases el salero puedo decirte “Dame el \$alero”. Aquí tenemos un error ortográfico (en programación se dice sintáctico) y será una forma incorrecta de pedirla. Igualmente, si quiero la sal y te digo Dame el azúcar también estaremos haciéndolo de forma incorrecta.

El intérprete de Python nos reñirá si escribimos “con faltas de ortografía”. Lo bueno es que nos indica dónde hemos cometido el error. Pero no siempre es fácil detectar dónde está nuestro error.

El inconveniente es si el intérprete de Python no detecta ningún error pero el programa no hace lo que queremos (acuérdate la frase que te dije en el punto 2.4)

Es lógico que muchas veces tengamos que hacer pruebas para comprobar que nuestro programa hace lo que debería y cómo debería.

Como el intérprete nos dio un error, vamos a corregirlo. Quizá sería interesante tener unas nociones de inglés, ya que muchos de los mensajes que nos muestre el intérprete serán en este idioma. Aunque no es estrictamente necesario.

Vemos que nos indica:

```
Consola x
>>> %Run -c $EDITOR_CONTENT
Traceback (most recent call last):
  File "<string>", line 1
    print hola mundo
    ^^^^^^^^^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
>>>
```

Thonny nos indica que hay un error sintáctico (hay algo mal escrito). En este caso nos dice que en la línea 1 (en este caso no hay otra) echa de menos unos paréntesis en la orden print. Incluso nos dice que si realmente queríamos decir print () pero no siempre será todo tan intuitivo.

Así que vamos a la parte superior y corregimos nuestro programa de la siguiente forma:

```
print('Hola Mundo')
```

Volvemos a pulsar F5 y obtenemos el siguiente resultado.



```
Consola x
>>> %Run -c $EDITOR_CONTENT
Hola Mundo
>>>
```

Lo que hace únicamente nuestro programa es escribir Hola Mundo en nuestra consola.

Ya podemos decir de forma orgullosa que somos programadores.

Aunque este programa únicamente nos muestra esa frase, no hace nada más. Pero todo esto irá cambiando conforme vayamos avanzando en las siguientes unidades.

Antes de pasar al siguiente tema, practicaremos un poco más, ya que es el modo en que más se aprende.

2.5 Recibiendo datos.

Ya sabemos hacer que el ordenador “hable” pero ¿podría el usuario contestarle en tiempo de ejecución? La respuesta es que sí, claro.

Para eso usamos la función INPUT.

Como es una función, ya sabes que debe ir seguida de unos paréntesis, así que escribiremos:

```
input()
```

No hay que escribir ningún espacio entre la t y el paréntesis.

Lo que hace esta función es aparezca en la pantalla un cursor parpadeante y la posibilidad de que el usuario escriba algo.

Pero claro, si el cursor se queda parpadeando no sabemos si el programa está esperando que escribamos algo, si está ejecutando instrucciones o que se ha quedado bloqueado, etc

Cuando escribamos programas, debemos hacerlos lo más amigables posible. Es decir, que parezca que el usuario siempre tenga el control de la situación.

Entonces, para mejorar la situación anterior, podríamos escribir lo siguiente:

```
input("dime tu nombre ")
```

Ojo al espacio en blanco después de la e. Cuando lo pruebes verás por qué.

Además, también debes aprender que Python es case sensitive, es decir, sensible a mayúsculas y minúsculas. Por lo que no es lo mismo escribir input que Input. Si lo ponemos en mayúsculas, el intérprete se quejará. Porque es una regla que hay que cumplir.

También me gustaría indicar que una instrucción en Python acaba cuando se pulsa Intro en la línea. Es decir, cuando escribimos print("Hola Mundo") y pulsamos Intro, se cambia a la siguiente línea y es una instrucción nueva.

Pero sigamos con el error anterior.

```
Traceback (most recent call last):  
  File "<string>", line 1, in <module>  
NameError: name 'Input' is not defined  
...
```

Básicamente, nos está diciendo que la orden Input no está definida.

Así que dejaremos nuestro programa de la siguiente forma:

```
print("dime tu nombre")
```

```
nombre = input()
```

Puede ser que no entiendas algo, pero esas ideas se desarrollan en la siguiente unidad. Como lo de qué pinta ahí la palabra nombre.

Ejecutamos nuestro programa (pulsando F5)

```
>>> %Run -c $EDITOR_CONTENT
Escribe tu nombre
José Ramón
```

Escribe la frase Escribe tu nombre y se espera con el cursor a que escribamos algo (podría ser nuestro nombre o lo que quisiéramos), lo escribimos, pulsamos Intro y ahí se acaba todo.

Ni nos saluda ni nada por el estilo. O sea, el programa no funciona mal, si no que quizá no hace lo que nosotros quisiésemos.

Si queremos que el ordenador nos salude podríamos escribir lo siguiente:

Ejecutamos y comprobamos que tampoco funciona bien, aunque no nos muestre ningún error (que no los hay), pero no hace lo deseado.

```
print('dime tu nombre')

nombre = input()

print('Hola nombre')
```

Programa

```
>>> %Run -c $EDITOR_CONTENT
dime tu nombre
José Ramón
Hola nombre
```

Resultado

Únicamente anticiparé que nombre es una variable (se estudia en la siguiente unidad) y guarda lo que escribamos, por lo que si queremos que funcione bien, debemos escribirlo de esta forma:

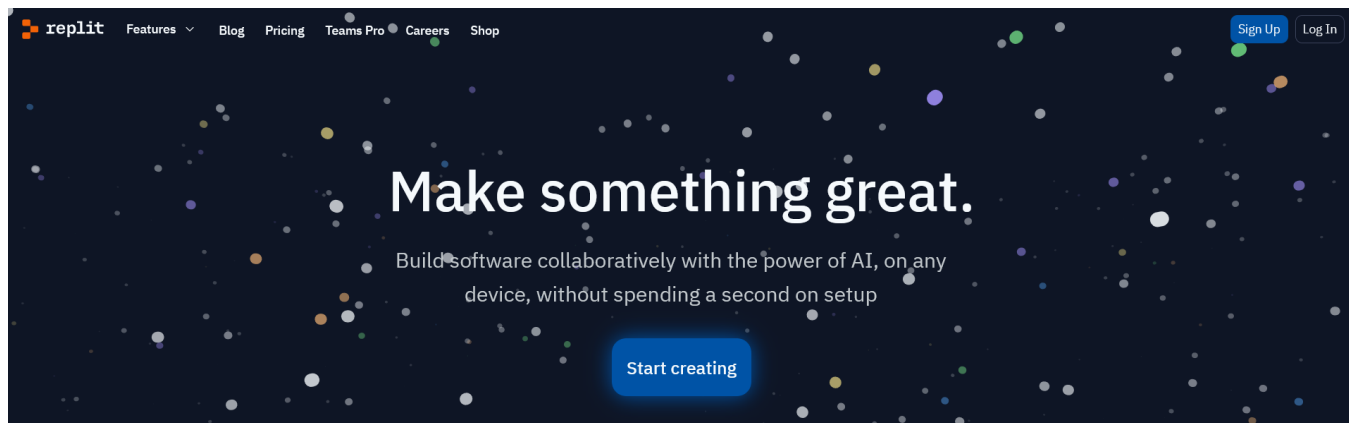
```
print('dime tu nombre')  
  
nombre = input()  
  
print('Hola ', nombre)
```

```
>>> %Run -c $EDITOR_CONTENT  
dime tu nombre  
José Ramón  
Hola  José Ramón
```

Por fin hace lo que nosotros queremos que haga.

2.6 Probando todo en replit.com

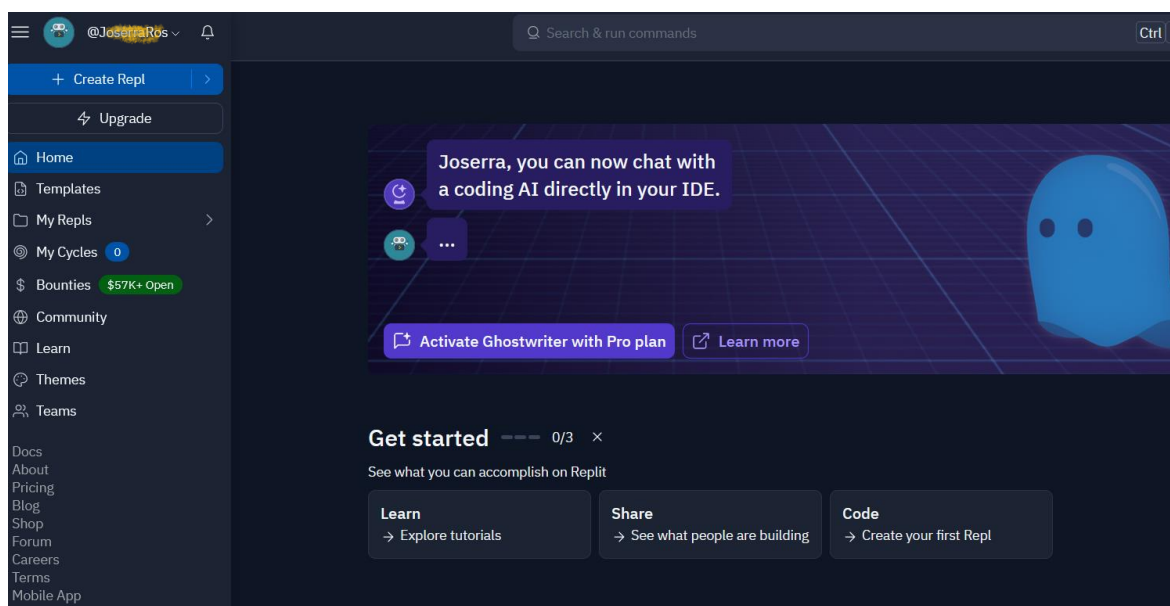
Como hemos comentado anteriormente, existen webs donde podemos escribir nuestros programas. Para este manual, probaremos replit.com

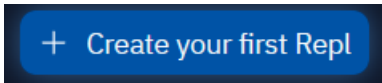


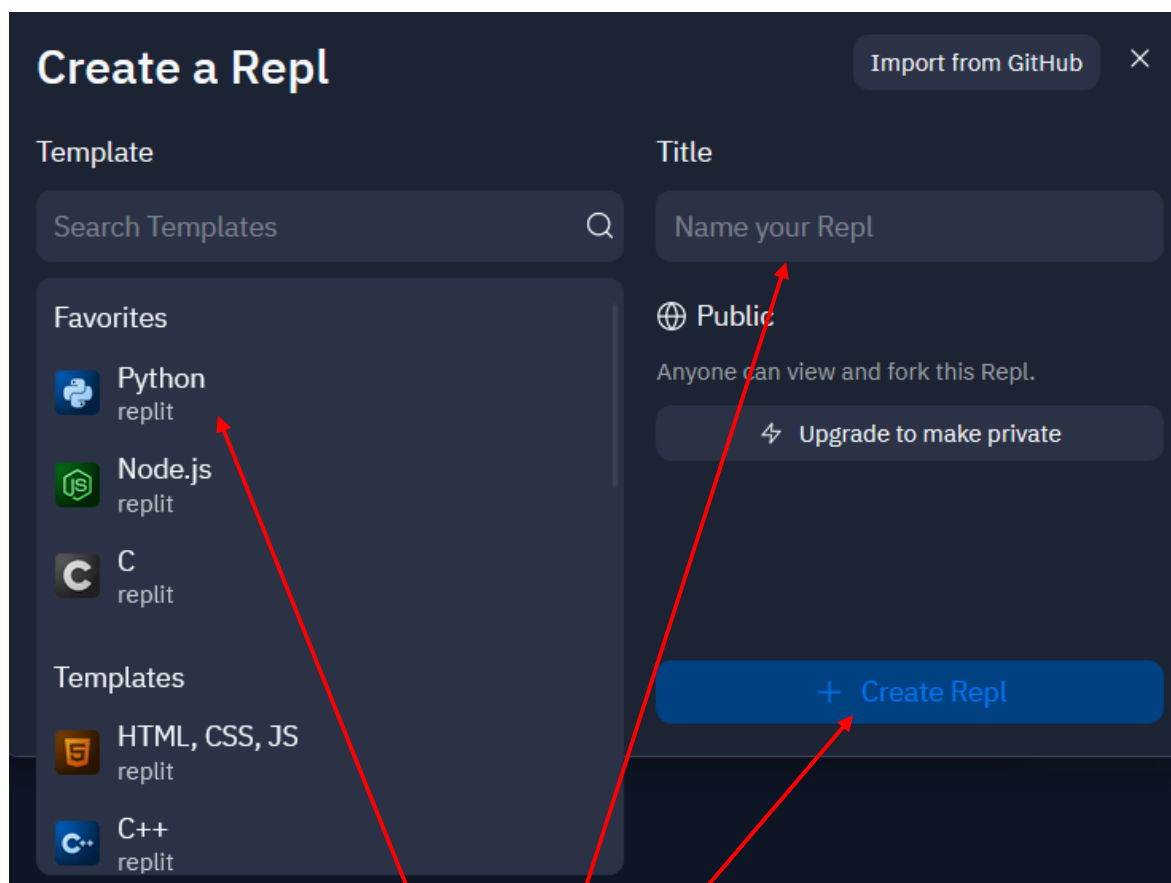
Éste es el aspecto que tiene la web.

Lo primero es hacer clic en Log In (arriba a la derecha) y entramos con nuestra cuenta de google, para hacerlo más sencillo. El único problema de este tipo de webs es que se encuentran en inglés. Aunque para probar nuestro ejemplo no necesitamos saber mucho.

Una vez que nos hemos logueado, veremos esta web:



Para programar nuestro ejemplo hacemos clic en My Repls (columna de la izquierda) , después en .



Ahora seleccionamos Python y en Title escribimos cómo se va a llamar nuestro programa. Y, por último, clic en + Create Repl

Cerramos la ventana de consejos y nos ponemos manos a la obra.

Para ello probaremos el programa que hemos escrito previamente.

En los siguientes pantallazos conocerás dónde se deben escribir los programas, cómo ejecutarlos o dónde veremos los resultados de su ejecución.

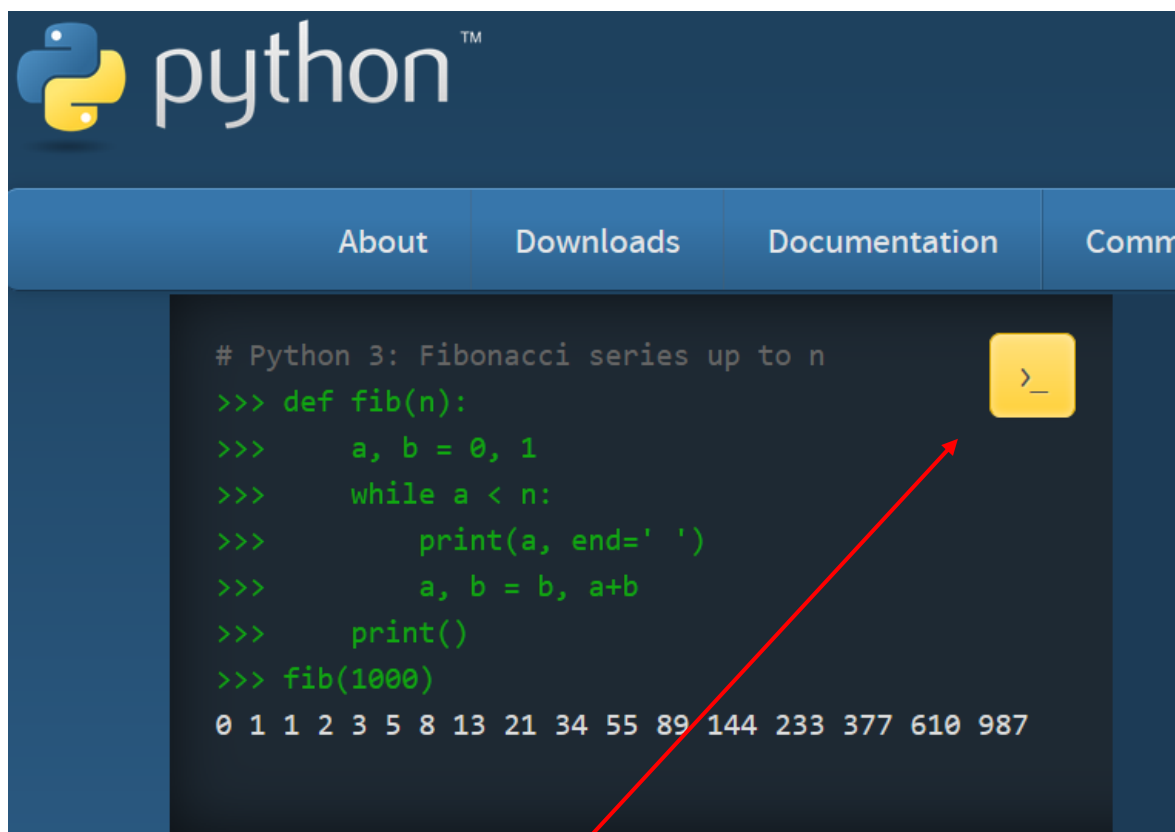
En la imagen anterior tenemos el resultado de nuestro programa.

En la parte derecha verás un cubo de basura, que nos permitirá borrar todo lo que aparece en la Consola.

Por motivos de comodidad, todos los ejemplos, ejercicios que realicemos y probemos a lo largo del curso, se probarán en el editor Thonny. Se deja a criterio del estudiante, probarlos en alguna web. No solo replit, ya que hay muchas para realizar lo mismo.

Pero replit.com no es la única web que nos permite programar en Python. En esta web tienes un listado de los mejores: [Intérpretes de Python online](#)

PythonAnywhere, Ideone.com, Codeanywhere, etc. Incluso Python tiene un Shell interactivo que también nos permite programar.



```
# Python 3: Fibonacci series up to n
>>> def fib(n):
>>>     a, b = 0, 1
>>>     while a < n:
>>>         print(a, end=' ')
>>>         a, b = b, a+b
>>>     print()
>>> fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
```

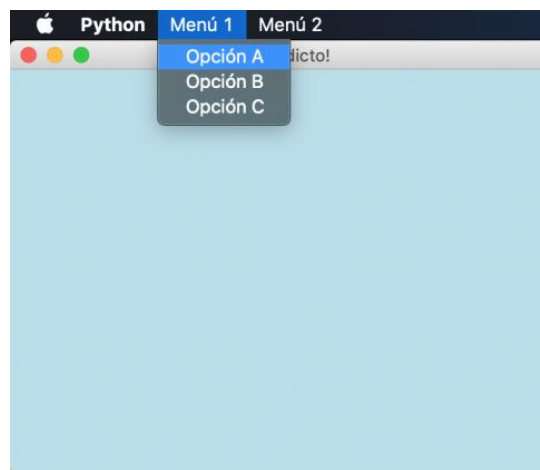
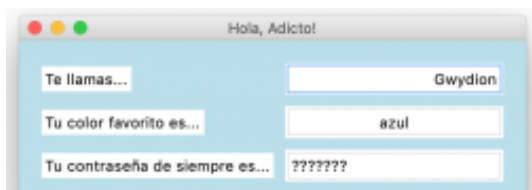
Si hacemos clic aquí nos aparecerá.

2.7 Python y más allá

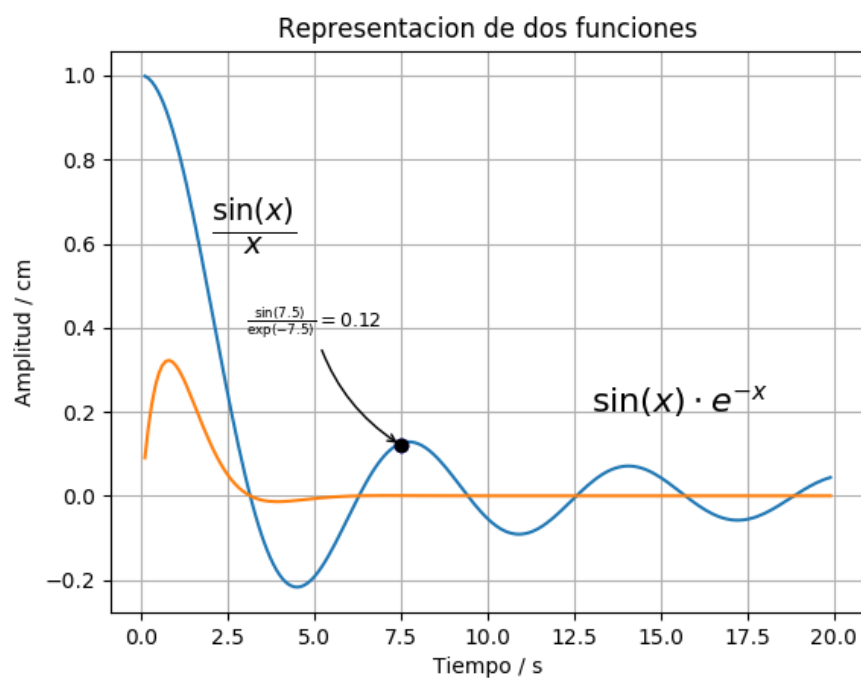
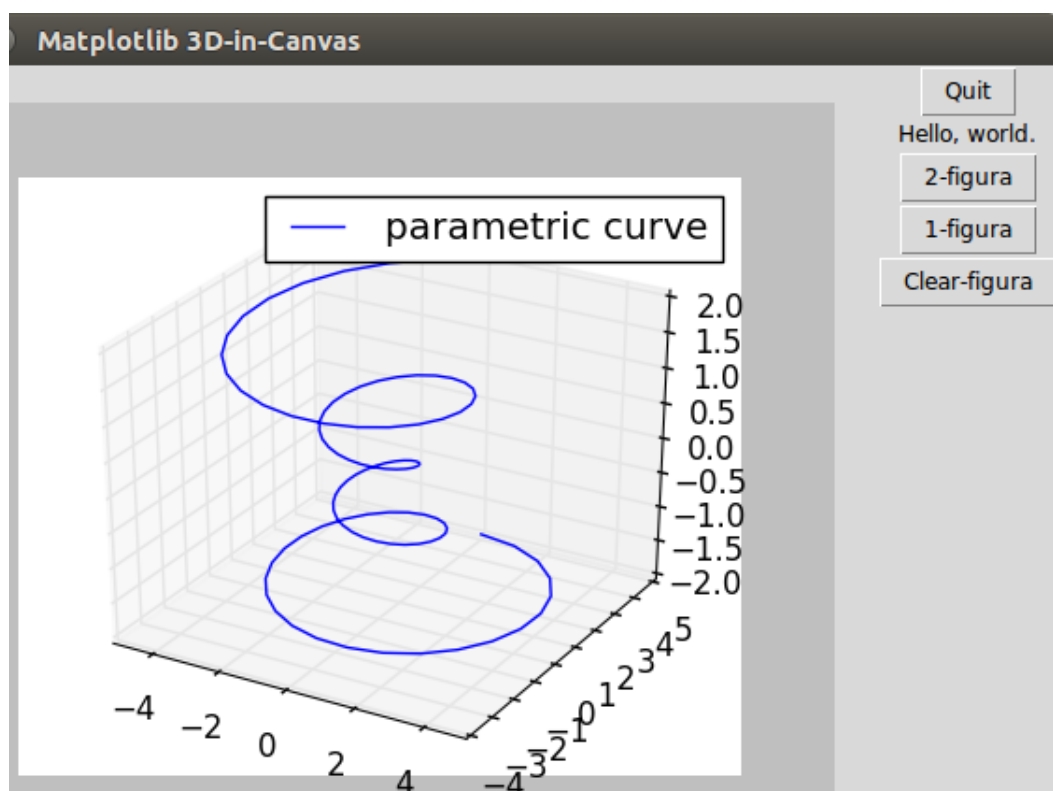
No sé si te habrás dado cuenta, pero desde que hemos comenzado a programar, sólo hemos utilizado lo que se llama modo consola. Es una pantalla que negra, que es la que se utilizaba en los comienzos de la informática (años 70).

Lo de programar con ventanas, botones (para Windows o Linux), etc se escapa de los conceptos de este curso. Existen herramientas de Python, como tkinter, que nos permiten crear los GUI (Graphic User Interface), es decir, las pantallas con las que interactuará el usuario. La creación de esto no es algo fácil ni trivial, por lo que haría falta un curso aparte para poder dominarlo.

Aquí tienes una serie de ejemplos creados con tkinter:



En el siguiente ejemplo, tenemos una gráfica matemática, algo para lo que se emplea mucho la programación en Python. Para ello existen paquetes (conjunto de programas), como pylab que se utilizan para hacer gráficos de funciones



3 Resumen

Podemos recordar lo que hemos aprendido en este tema:

- Python es un lenguaje de programación interpretado.
- Hace falta utilizar el intérprete de python y algún editor.
- Podemos cometer errores de diversos tipos a la hora de programar.
- Hay multitud de editores para crear nuestro programa.
- Sabemos cómo mostrar datos y que el programa le pida datos al usuario.

En la siguiente Unidad ya comenzamos a estudiar de manera progresiva las diferentes partes del Lenguaje Python. Que serán el 90% de nuestros programas, como las variables o las estructuras de control de flujo.

Vamos a ello.



¡Enhorabuena!

Has finalizado el estudio de esta unidad



comunicación@bilib.es